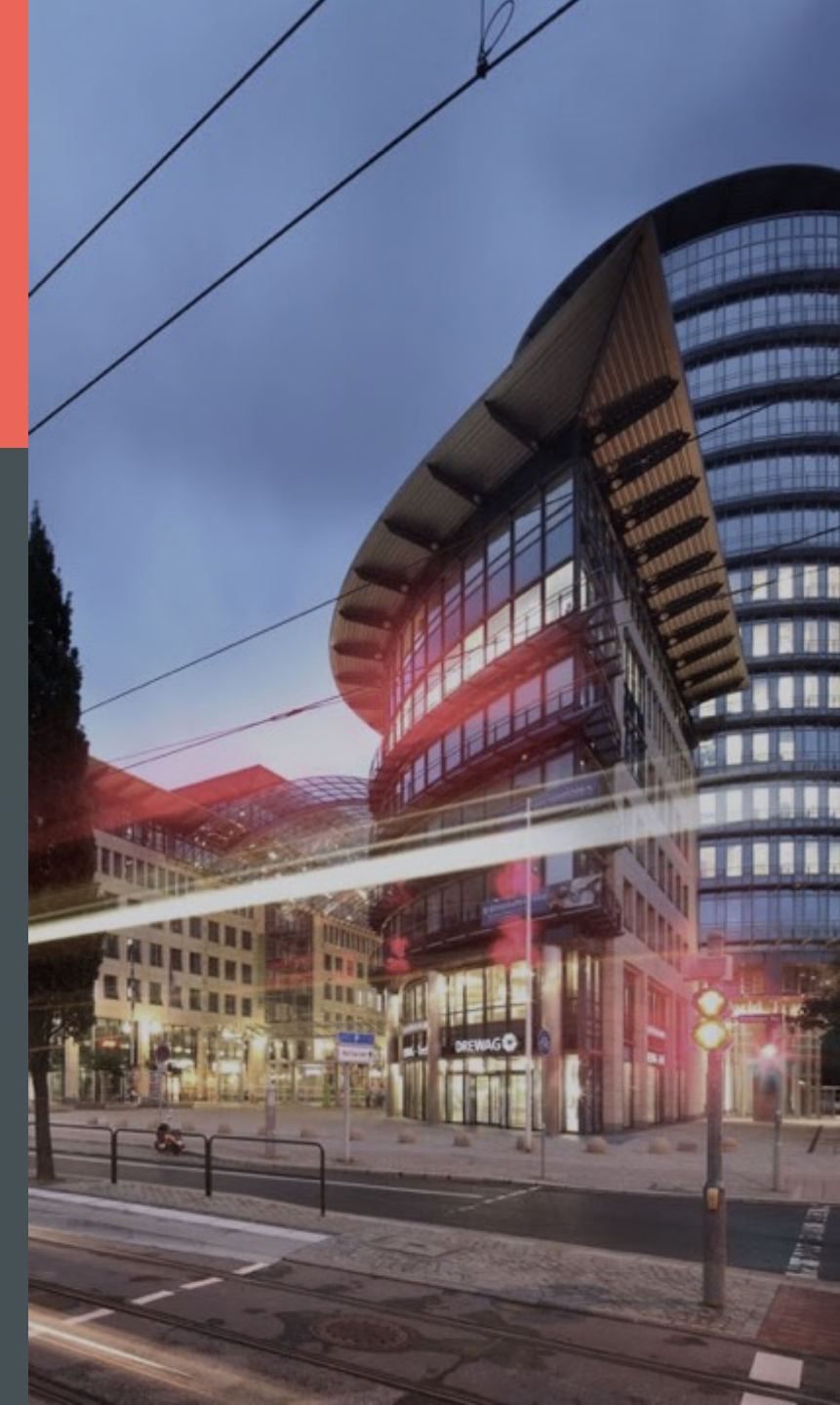


Oracle Advanced Compression



Erfahrungen aus dem praktischen Einsatz



Otto Group Solution Provider (OSP)



Gründung:
März 1991

Muttergesellschaft:
Otto Group

Standorte:
Dresden, Hamburg, Altenkunstadt, Madrid, Taipei

Anzahl Mitarbeitende:
> 500

Geschäftsführer:
Dr. Stefan Borsutzky, Norbert Gödicke

Zur Person



Peter Ramm

Software Architekt / Teamleiter bei OSP in Dresden

> 30 Jahre Historie in IT-Projekten

Schwerpunkte:

- Entwicklung von OLTP-Systemen auf Basis von Oracle-Datenbanken
- Architektur-Beratung bis Trouble-Shooting
- Performance-Optimierung bestehender Systeme

Mail: Peter.Ramm@ottogroup.com

Kompressionsverfahren der Oracle-DB

Komprimierungsverfahren der Oracle-DB in dieser Session:

- Table Compression
- Index Compression
- LOB Compression

Weitere Komprimierungen sind verfügbar z.B. für:

- InMemory Compression
- Data Pump Exports
- RMAN Backup Compression

Beispiele unter Nutzung des Analyse-Tools Panorama (<https://github.com/rammpeter/panorama>)

Kompressionsverfahren der Oracle-DB - Tabellen

Verfahren	Support für DML	Advanced Compression Option	Kommando-Syntax
Basic compression	Nein	Nein	ROW STORE COMPRESS BASIC
Advanced Compression	Ja	Ja	ROW STORE COMPRESS ADVANCED bzw. COMPRESS FOR OLTP (12.x)
HCC Query Compression	Nein	Ja	COLUMN STORE COMPRESS FOR QUERY (LOW HIGH)
HCC Archive Compression	Nein	Ja	COLUMN STORE COMPRESS FOR ARCHIVE (LOW HIGH)

Übliche Kompressionsraten für Basic und Advanced Compression: 1:2 .. 1:4

Auf Engineered Systems (Exadata/ODA) bzw. ZFS-Appliance sind per HCC damit Kompressionsraten realistisch:

- 1:10 mit `COMPRESS FOR QUERY HIGH` z.B. für eine Buchungshistorie mit 55 Spalten
- 1:64 mit `COMPRESS FOR ARCHIVE LOW` z.B. für eine Bestandshistorie mit 134 Spalten

Komprimieren von existierenden Tabellen

Kommando	Wirkung
ALTER TABLE .. <compress>;	Komprimieren aller weiteren DML-Operationen
ALTER TABLE .. MOVE <compress>;	Komprimieren der vorhandenen Daten + bei weiterem DML. DML wird geblockt während MOVE, Indizes werden invalid.
ALTER TABLE .. MOVE PARTITION <compress> UPDATE INDEXES;	Komprimieren der vorhandenen Daten + bei weiterem DML. DML wird geblockt während MOVE. Indizes werden gepflegt, aber Index-Partitionen werden Unusable während des MOVE.
ALTER TABLE .. MOVE TABLE / PARTITION <compress> ONLINE;	Komprimieren der vorhandenen Daten + bei weiterem DML. DML ist möglich während MOVE, Indizes werden gepflegt und bleiben nutzbar während des MOVE.
ALTER TABLE .. MODIFY DEFAULT ATTRIBUTE <compress>;	Komprimieren bei DML für neue Partitionen, vorhandene bleiben unverändert.
Komprimierung per Online Redefinition	Bislang keine Erfahrung
Automatic Data Optimization (ADO) Link zu Whitepaper	Storage tiering and heat map / Bislang keine eigene Erfahrung Aber vielversprechend für minimalen Handling-Aufwand

Teil-Komprimierung von partitionierten Tabellen

- Oft lässt sich eine Altersstruktur von Daten einer Tabelle über Intervall-Partitionierung abbilden z.B.:

```
PARTITION BY RANGE (DATUM) INTERVAL (NUMTODSINTERVAL(1,'DAY'))  
(PARTITION MIN VALUES LESS THAN  
(TO_DATE(' 2013-01-01', 'YYYY-MM-DD', 'NLS_CALENDAR=GREGORIAN')))
```

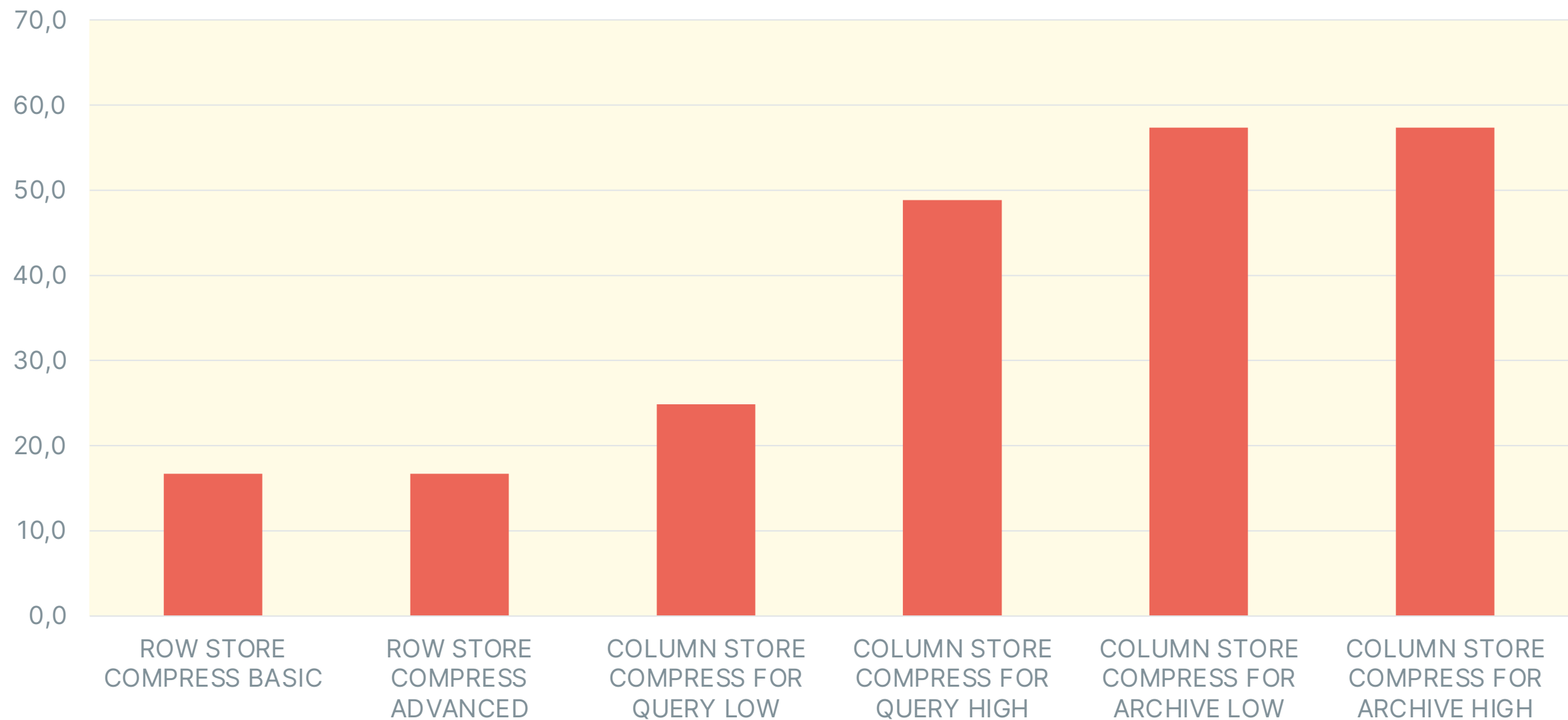
- Damit Housekeeping von Altdaten in Sekunden per `ALTER TABLE DROP PARTITION` statt `DELETE`
- Nicht mehr DML-aktive Partitionen können dann komprimiert werden.
Beispiel: einzelne Partition mit 682.000 Records, 55 Spalten, 10 globale Indizes, 115 MB unkomprimiert

Aktion	Nonparallel	Parallel (64)	Size	Faktor
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS;</code>	5 Sek.	2 Sek.	36,6 MB	3,1
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS ADVANCED;</code>	5 Sek.	1,3 Sek.	36,6 MB	3,1
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS FOR QUERY HIGH;</code>	7 Sek.	2 Sek.	13,4 MB	8,6
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS FOR QUERY HIGH UPDATE INDEXES;</code>	43 Sek.	2,7 Sek.	13,4 MB	8,6
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS FOR QUERY HIGH ONLINE;</code>	3353 Sek.	340 Sek.	13,4 MB	8,6
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS FOR ARCHIVE LOW;</code>	9 Sek.	0,6 Sek.	12,8 MB	9,0
<code>ALTER TABLE .. MOVE PARTITION .. COMPRESS FOR ARCHIVE HIGH;</code>	12 Sek.	0,8 Sek.	11,6 MB	9,9

Resultate eines Vergleiches mit partitionierter Tabelle

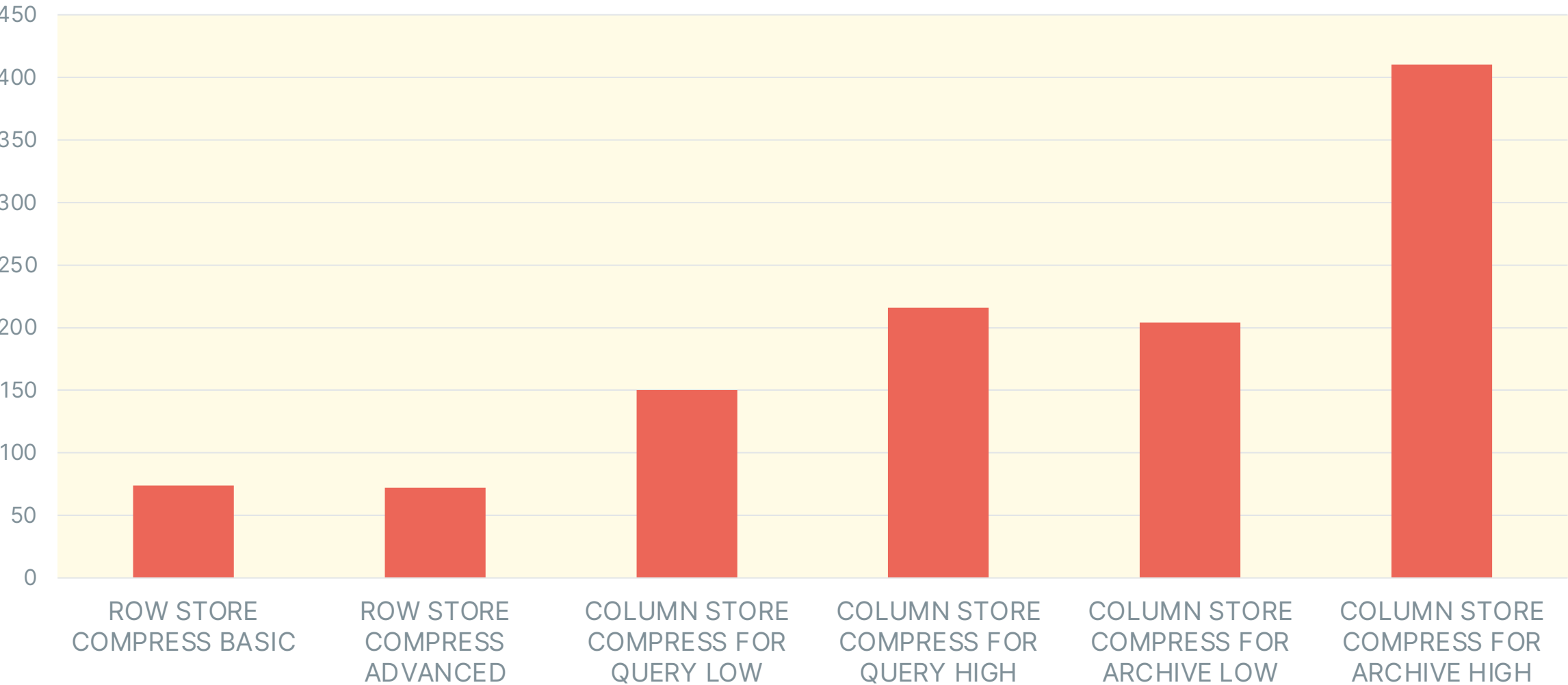
- Tabelle mit 14 Mrd. Records, 134 Spalten und > 3000 Partitionen
- Eine betrachtete Partition mit 8 Mio. Records
- Zugriff auf Daten einer Partition bei vollständigem Caching der Tabelle in Memory
 - RowID-Zugriff auf alle Spalten eines einzelnen Record
 - Full Table Scan über gesamte Partition mit Zugriff auf alle Spalten sowie auf nur 2 von 134 Spalten
- Zugriff auf Daten einer Partition bei Direct Path Read ohne Nutzung des Buffer Cache
- Getestet auf Exadata X6-2L Extreme Flash
- Test aus https://github.com/rammpeter/oracle_benchmarks

Komprimierungsfaktor einer Partition mit ca. 8 Mio. Records



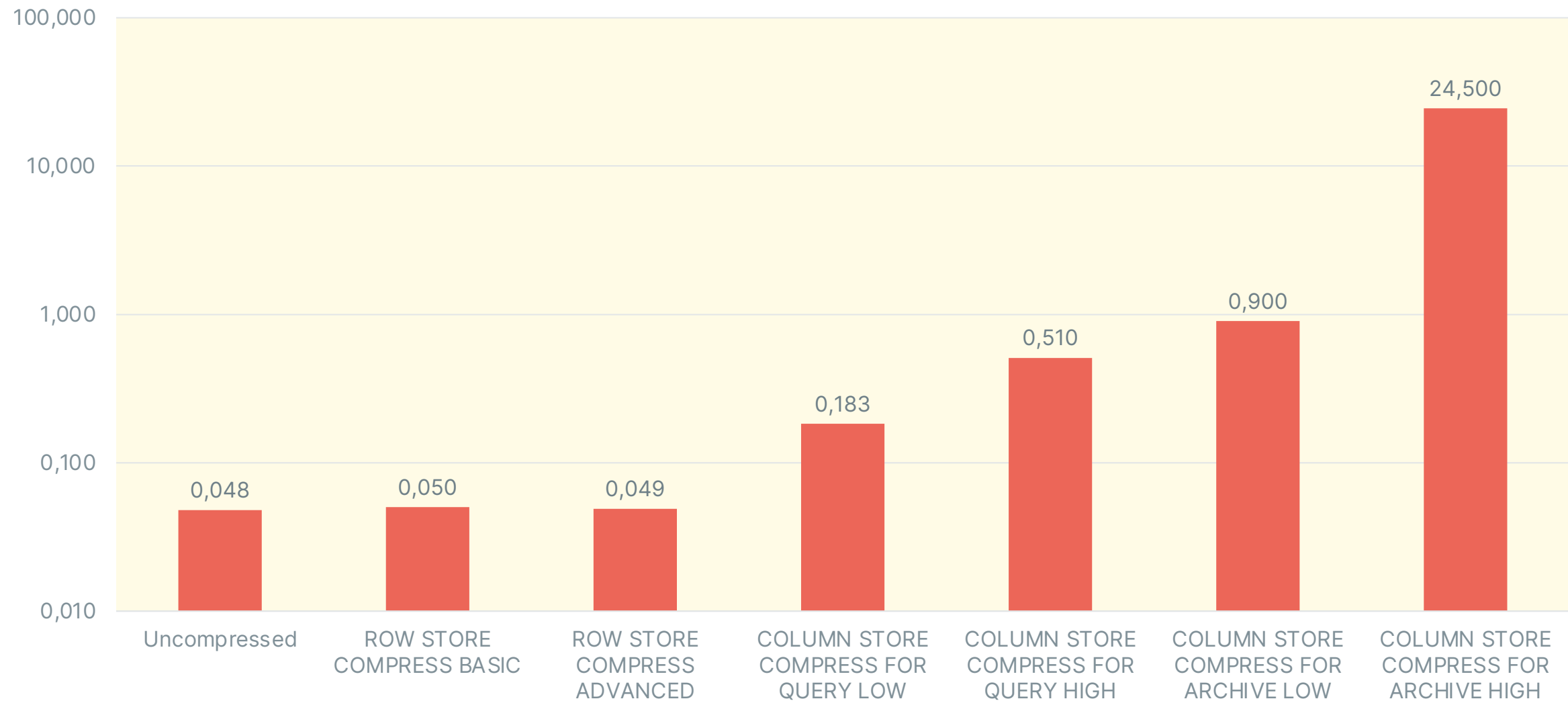
Komprimierungszeit (Sek.) einer Partition

```
ALTER TABLE .. MOVE PARTITION .. <Compress Clause> NOPARALLEL;
```



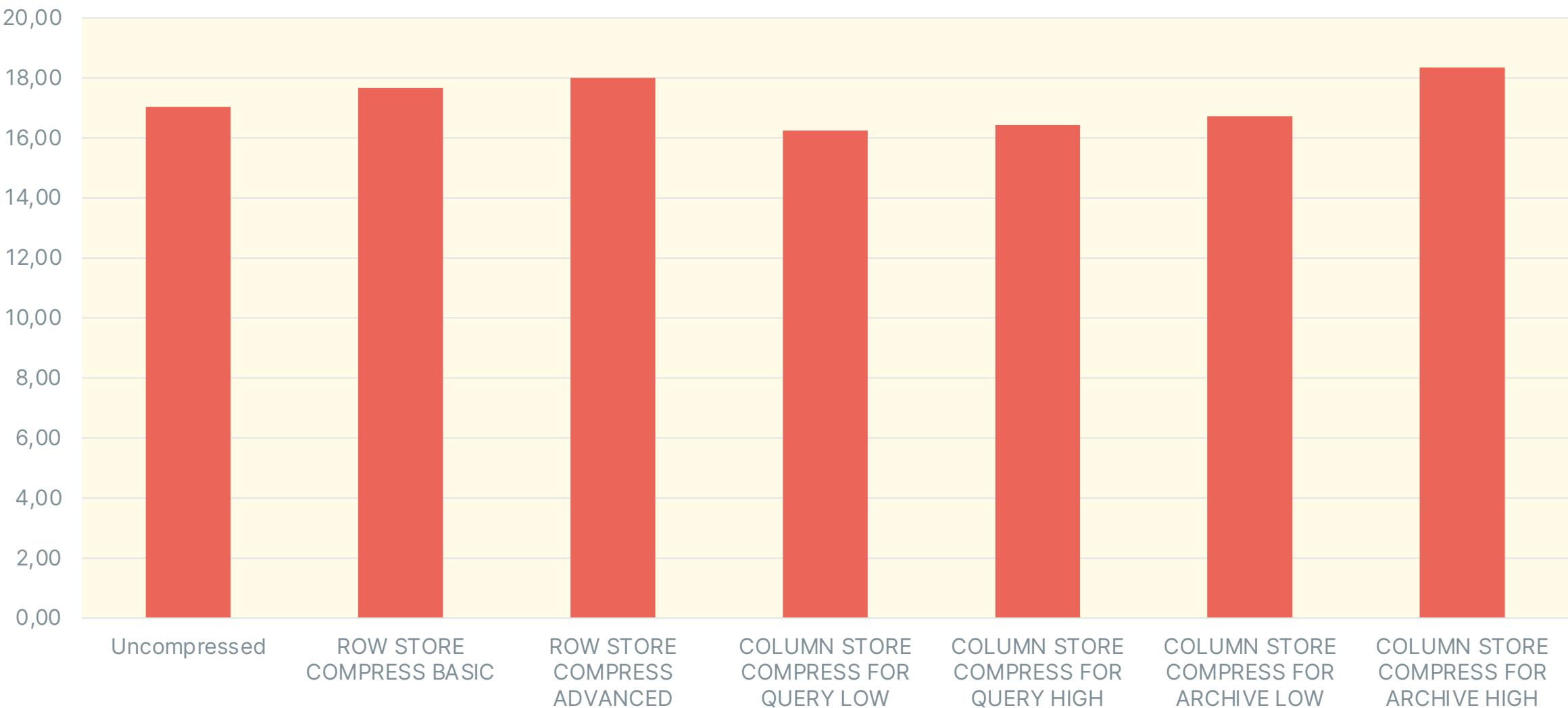
Zugriffszeit per RowID (ms) auf einer Partition

Zugriff auf alle 134 Spalten eines Records



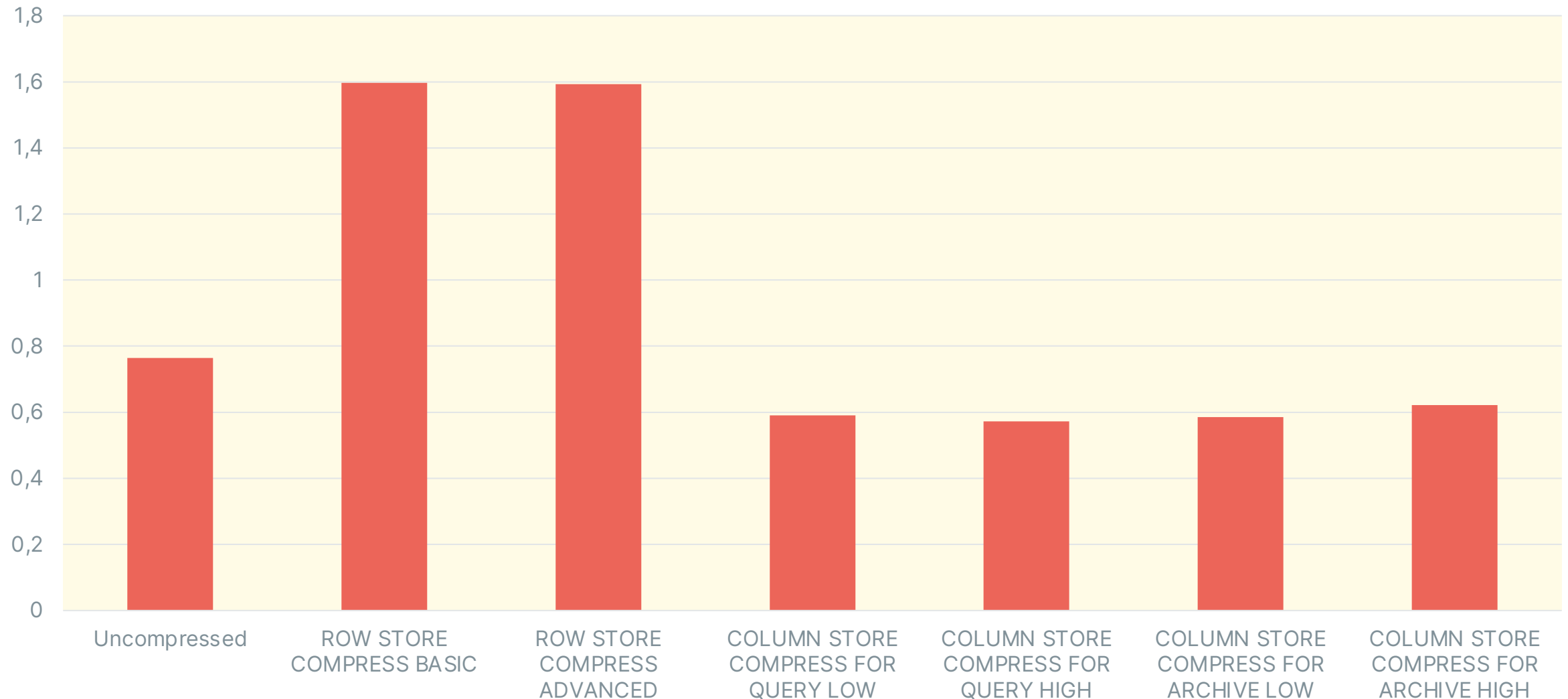
Laufzeit für Full Scan mit allen Spalten einer Partition (Sek.)

Alle 134 Spalten gefetcht von Buffer Cache (keine physical reads) für die ersten 1 Mio. Records



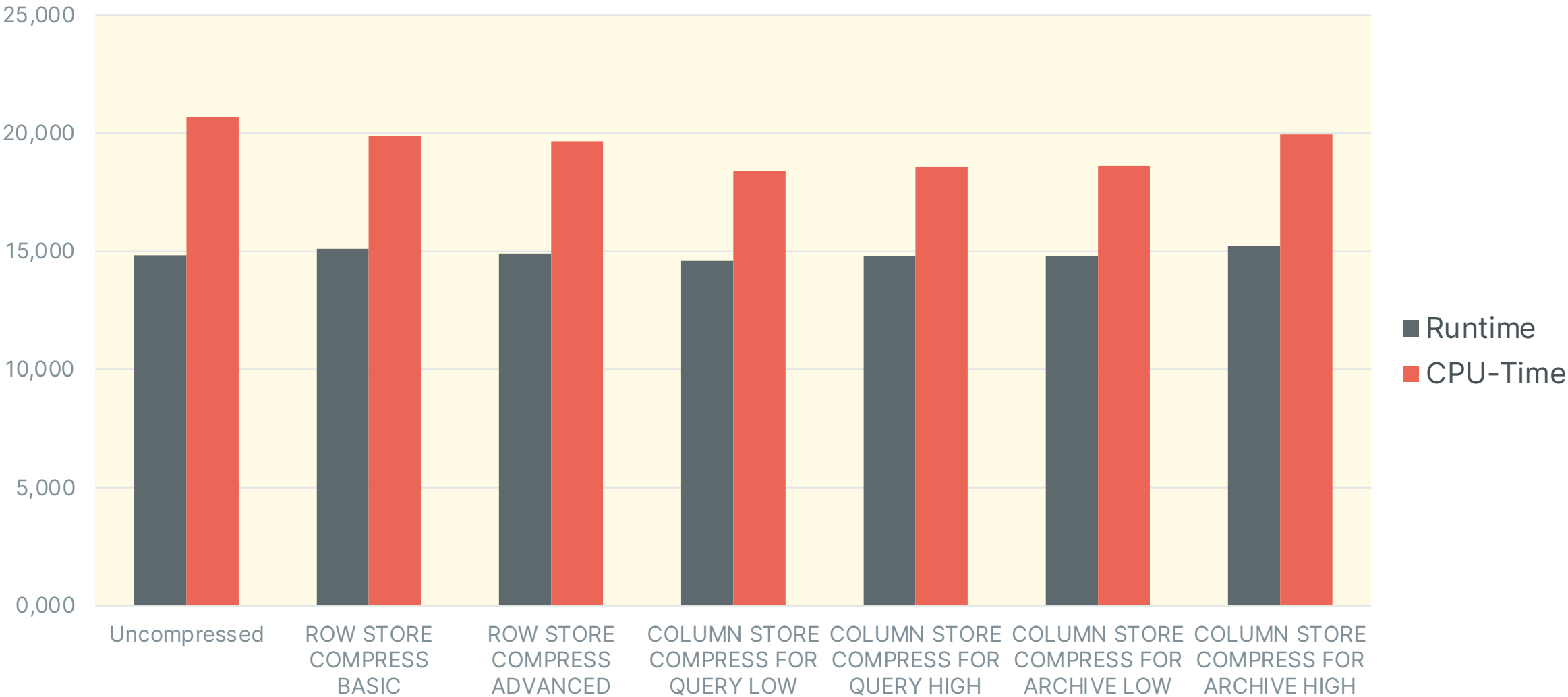
Laufzeit für Full Scan mit nur 2 Spalten einer Partition (Sek.)

Nur 2 von 134 Spalten gefetcht von Buffer Cache für die ersten 1 Mio. Records (keine physical reads)



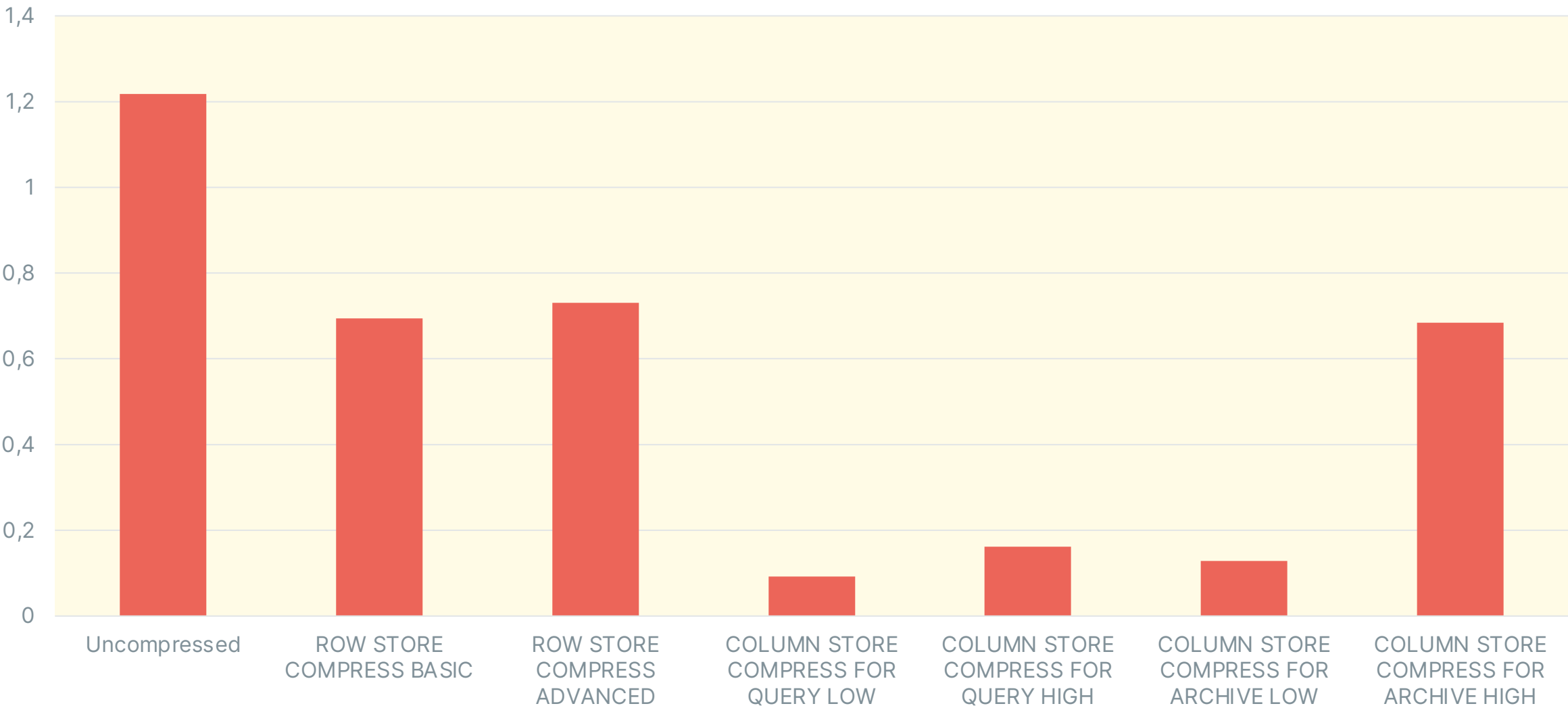
Laufzeit für Full Scan direct aller Spalten einer Partition (Sek.)

Full Scan per Direct Path Read mit Cell Server Offload,
1 Mio. Records mit allen 134 Spalten gefetcht, Parallel Degree = 2



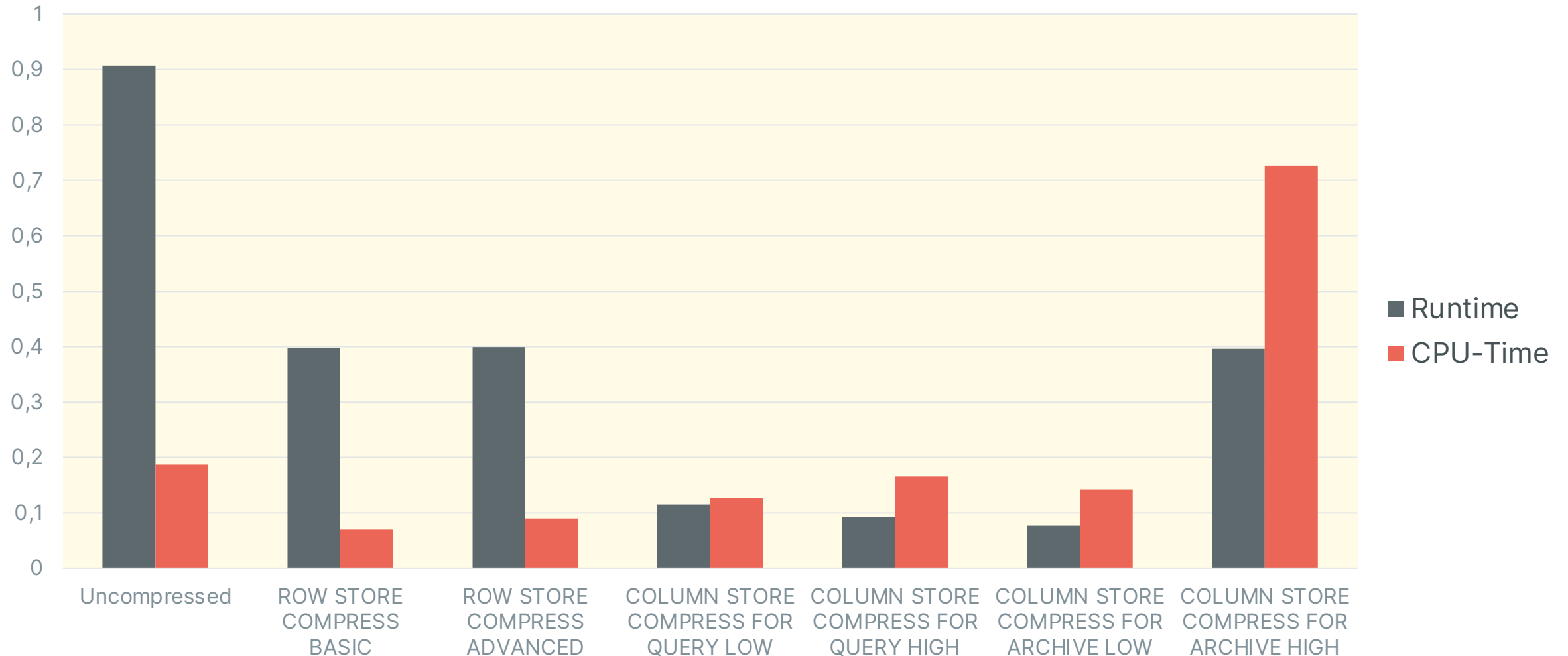
Laufzeit für Filterung von Buffer Cache einer Partition (Sek.)

Full Scan mit Filterbedingung auf 2 Spalten einer Partition und Zugriff auf Buffer Cache (keine physical reads)



Laufzeit für Filterung mit Direct Path Read einer Partition (Sek.)

- Full Scan mit Filterbedingung auf 2 Spalten einer Partition, Direct Path Read **ohne** Cell Server Offload
- Cell smart table scan mit Filterung und Komprimierung läuft aktuell buggy 6 Sekunden (Grid: 19.19, DB 19:18)



Nutzung von COMPRESS ADVANCED mit Updates

- Mit den Releases 12.x und 18.x gab es Probleme in Kombination von COMPRESS FOR OLTP und intensiven Update DML-Statements, dargestellt mit Testcase in diesem [Blog Post](#)
- Bei Update wurden nach Dekomprimierung nicht mehr in den ursprünglichen Block passende Rows verschoben in neu allokierte Überlaufblöcke (Migrated Rows)
- Diese Rows verblieben dann im migrierten Zustand, auch wenn im ursprünglichen Block wieder wurde
- Im Extremfall wuchs damit die Größe einer komprimierten Tabelle deutlich über die Größe im unkomprimierten Zustand
- RowID-Zugriff auf eine einzelne Row erforderte mehrere Blockzugriffe (consistent gets) statt einem.
- Insert und Delete-Operationen zeigten dieses Verhalten nicht bzw. deutlich weniger
- Mit Rel. 19 (getestet mit 19.18) tritt dieses Verhalten zwar noch sporadisch auf, aber mit drastisch geringerem Risiko als in Rel. 12.x
- Fazit: bei COMPRESS ADVANCED von Tabellen mit signifikanter Menge an Update-DML sollte die Größe bzw. Relevanz von migrated rows im Blick behalten werden

Fallstrick bei Komprimierung von partitionierten Tabellen

- Das Kommando

```
ALTER TABLE <tablename> COMPRESS ...;
```

ändert sowohl die Vorgabe der Komprimierung für neue Partitionen als auch den Kompressions-Status der existierenden Tabelle bzw. Partitionen. Dies allerdings ohne die physischen Daten wirklich zu komprimieren.

- Die Anpassung der Komprimierungsvorgabe für neue Partitionen sollte geschehen per

```
ALTER TABLE <tablename> MODIFY DEFAULT ATTRIBUTES COMPRESS ...;
```

In diesem Falle wird nur die Vorgabe für neue Partitionen angepasst, der Komprimierungsstatus der bestehenden Partitionen bleibt unverändert.

- Dictionary-Informationen:

- user_tables.COMPRESSION, user_tables.COMPRESS_FOR
- user_part_tables.DEF_COMPRESSION, user_part_tables.DEF_COMPRESS_FOR

Kompressionsverfahren der Oracle-DB - Indizes

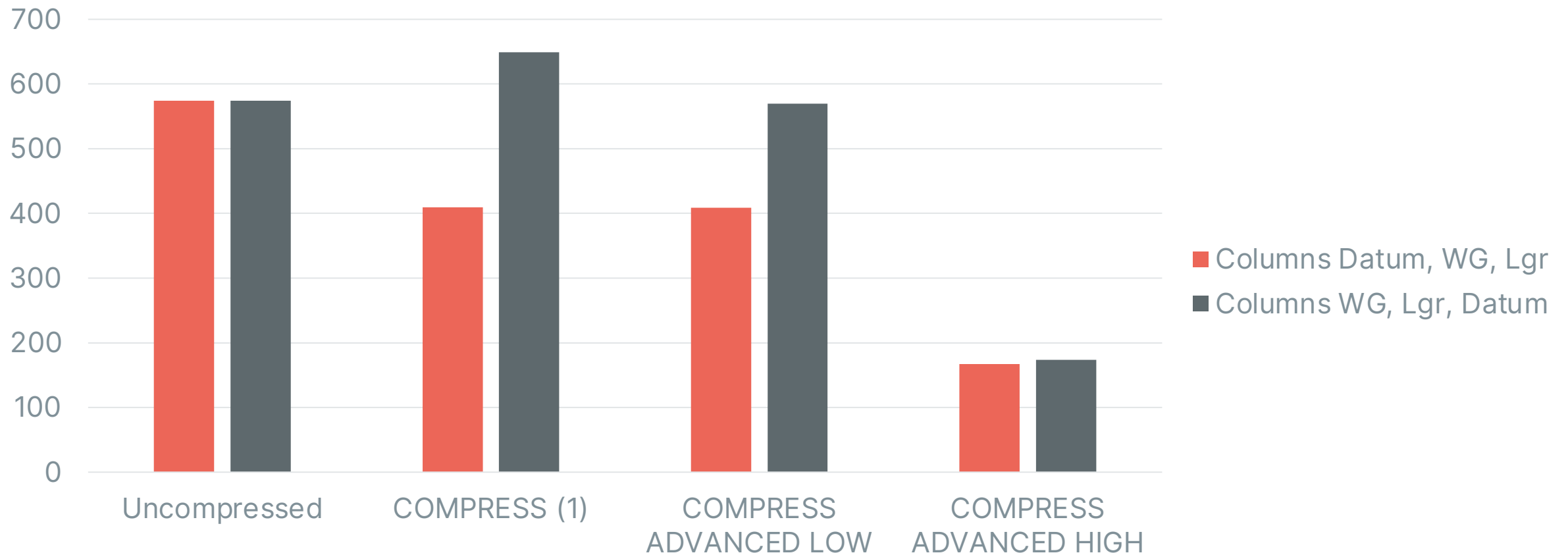
Verfahren	Advanced Compression Option notwendig	Kommando	Beschreibung
Prefix Key Compression	Nein	COMPRESS <n>	Deduplizierung identischer Spaltenwerte innerhalb eines Index-Blocks, optional Vorgabe der Anzahl zu komprimierender Spalten von links beginnend
Advanced Compression Low	Ja	COMPRESS ADVANCED LOW	Analog Prefix Key Compression, aber mit automatischer Kalkulation der optimalen Prefixlänge je Block
Advanced Compression High	Ja	COLUMN ADVANCED HIGH	Kombination verschiedener Verfahren mit deutlich größerer Wirkung als Key Compression. Nicht für Bitmap, IOT und function based Indexes.

Resultate eines Vergleiches mit partitioniertem Index

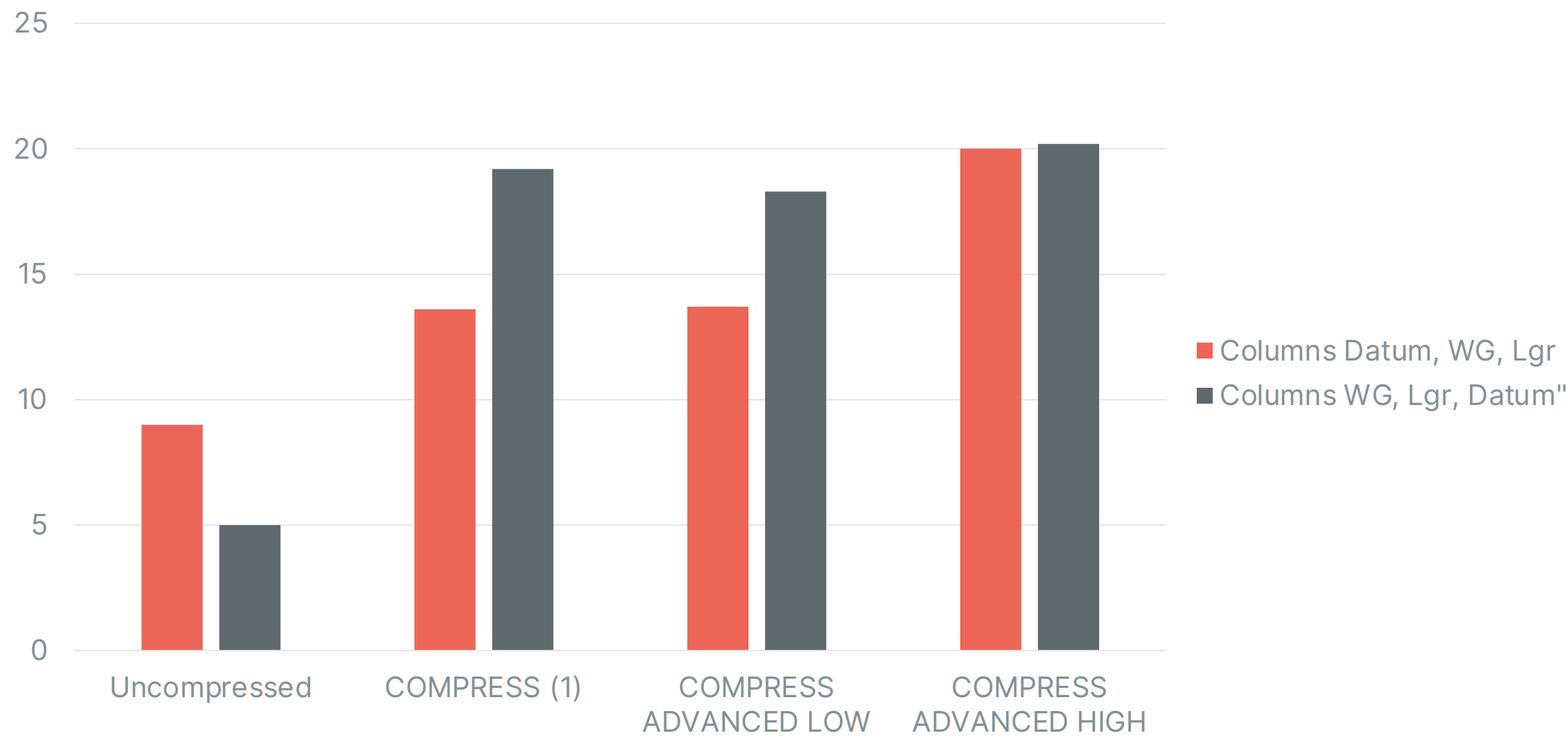
- Tabelle mit 14 Mrd. Records, 134 Spalten und > 3000 Partitionen, Intervall-partitioniert nach Datum
- Unique Index aus 3 Spalten:
 - DATUM (DATE, durchschnittlich 8 Byte) mit 3049 eindeutigen Werten
 - WARENGRUPPE_ID (NUMBER, durchschnittlich 7 Byte) mit 7 Mio. eindeutigen Werten
 - LGR_BEREICH_ID (NUMBER, durchschnittlich 4 Byte) mit 30 eindeutigen Werten
- Test Index Range Scan bzw. Index Skip Scan auf Warengruppe_ID, Lgr_Bereich_ID (2365 Rows im Result)
Ein Zugriff für jede Partition
- Test Index Unique Scan mit allen drei Spalten, Zugriff nur in einer Partition
- Test aus https://github.com/rammpeter/oracle_benchmarks

Größe des partitionierten Index in GB

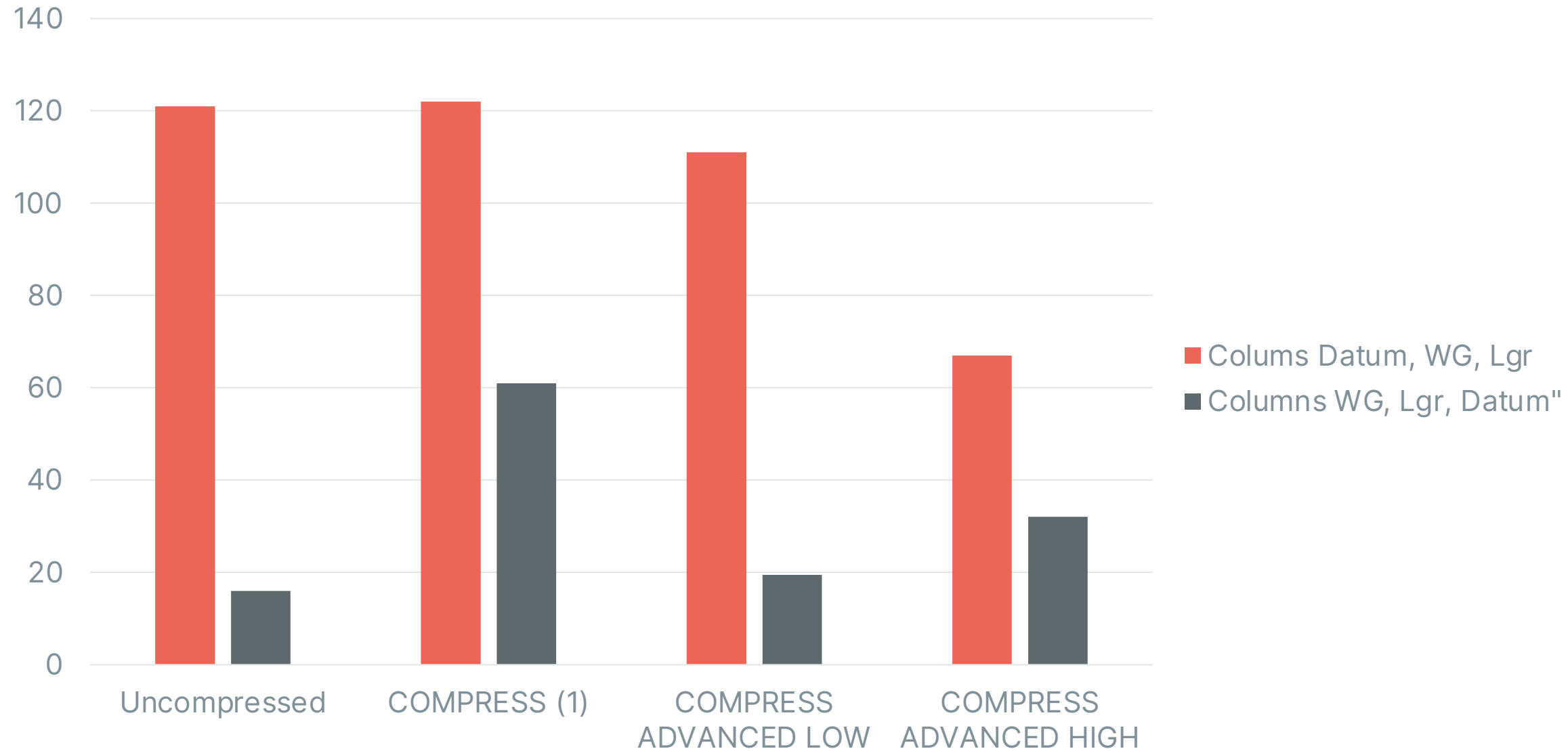
- Partitionskriterium als führende Spalte beeinflusst deutlich die Wirkung der Komprimierung
- Identische Werte bei Partitionierungskriterium als erste Spalte führen zu deutlich effektiverer Prefix Key Compression
- Für **ADVANCED HIGH** spielt die Position der Spalten mit größter Redundanz keine Rolle



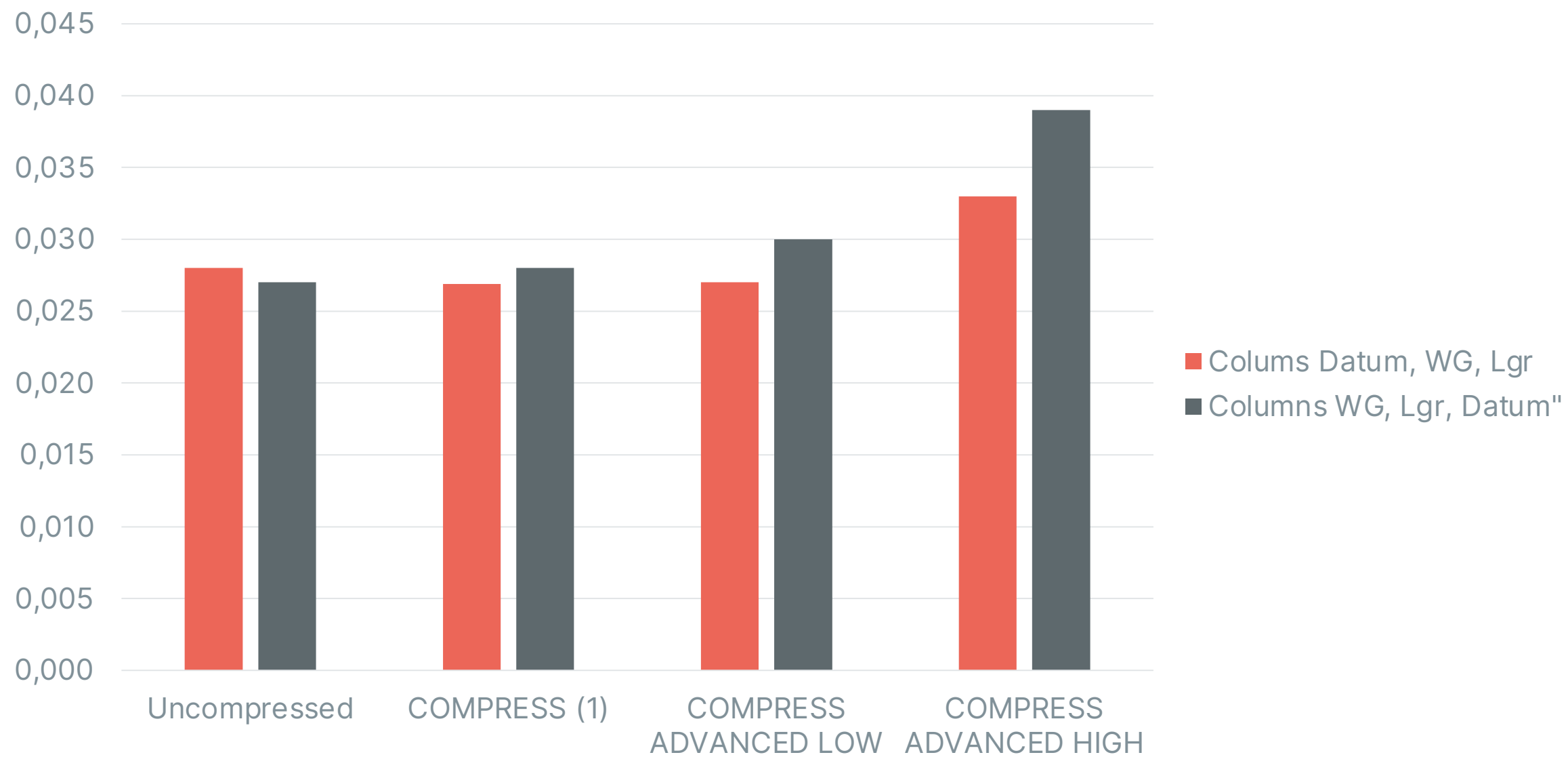
Erstellungszeit des partionierten Index in Minuten



Zugriffszeit bei Range Scan bzw. Skip Scan je Partition in ms

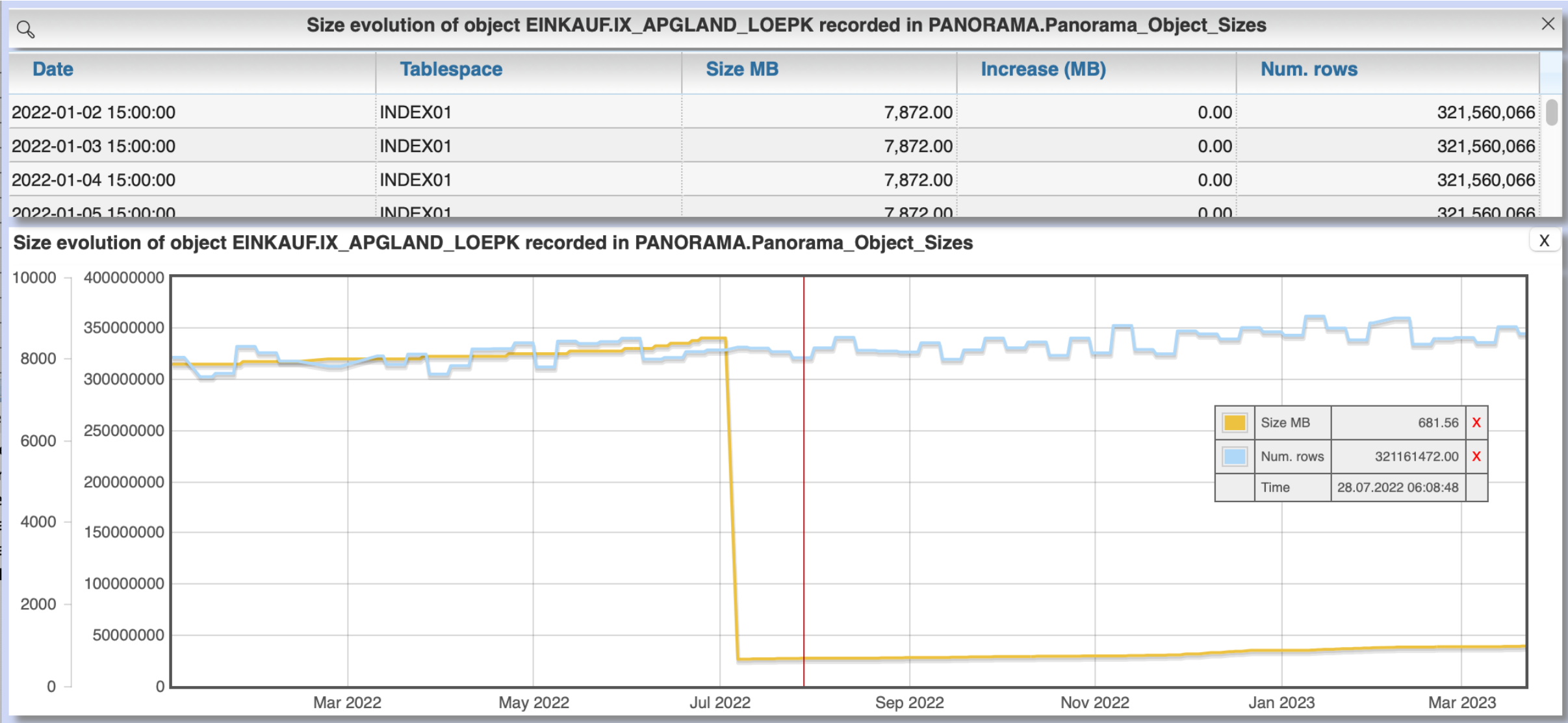


Zugriffszeit bei Unique Scan in ms



Beispiel für Größenentwicklung eines Index

Index mit nur 144 distinct keys vor und nach COMPRESS ADVANCED HIGH



Probleme im Zusammenhang mit Index-Komprimierung

- LOCAL partitionierter primary Key Index einer partitionierten Tabelle, COMPRESS ADVANCED HIGH
 - ALTER INDEX ... MODIFY PARTITION ... SHRINK SPACE COMPACT;
 - ORA-00600: internal error code, arguments : [6302], [1], [], [], [], [], [], [], [], [], [], []
 - Das Problem tritt mehrfach auf bei partitionierten Indizes, jedoch nicht deterministisch
 - Klärung mit Oracle-Support bislang nicht erfolgreich
- Für Indizes mit COMPRESS ADVANCED HIGH werden im Optimizer für den Zugriff so exorbitant kleine Kosten ausgewiesen, dass deutlich passendere mehrspaltige Indizes mit gleichen führenden Spalten nicht genutzt werden.
 - Spaltenreihenfolge des komprimierten Index so wählen, dass die führenden Spalten redundanzfrei nur in diesem Index vorkommen

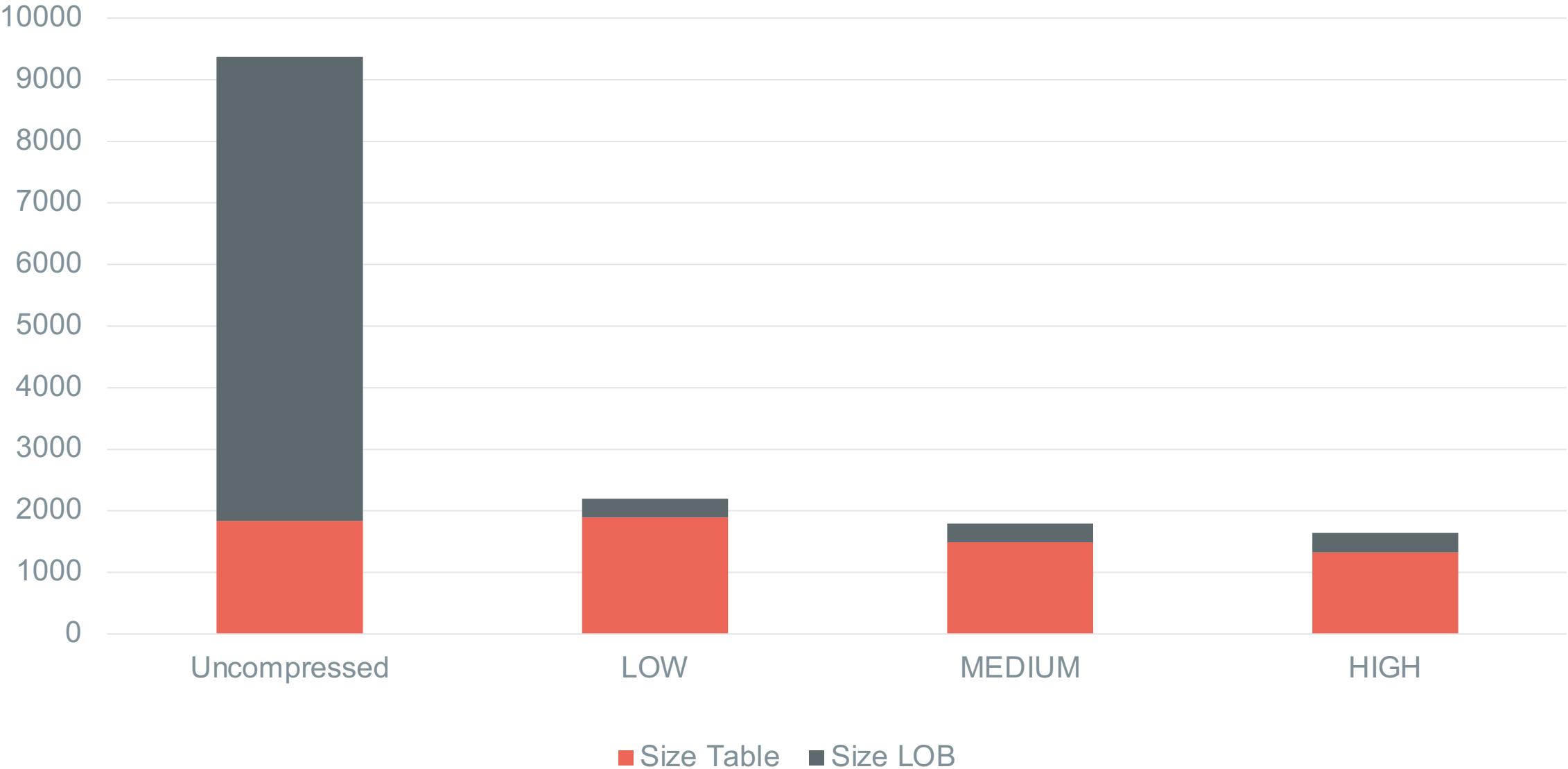
Kompressionsverfahren der Oracle-DB - LOBs

Verfahren	Advanced Compression Option notwendig	Kommando
LOW	Ja	LOB (<Column>) STORE AS SECUREFILE (COMPRESS LOW)
MEDIUM (Default)	Ja	LOB (<Column>) STORE AS SECUREFILE (COMPRESS MEDIUM)
HIGH	Ja	LOB (<Column>) STORE AS SECUREFILE (COMPRESS HIGH)

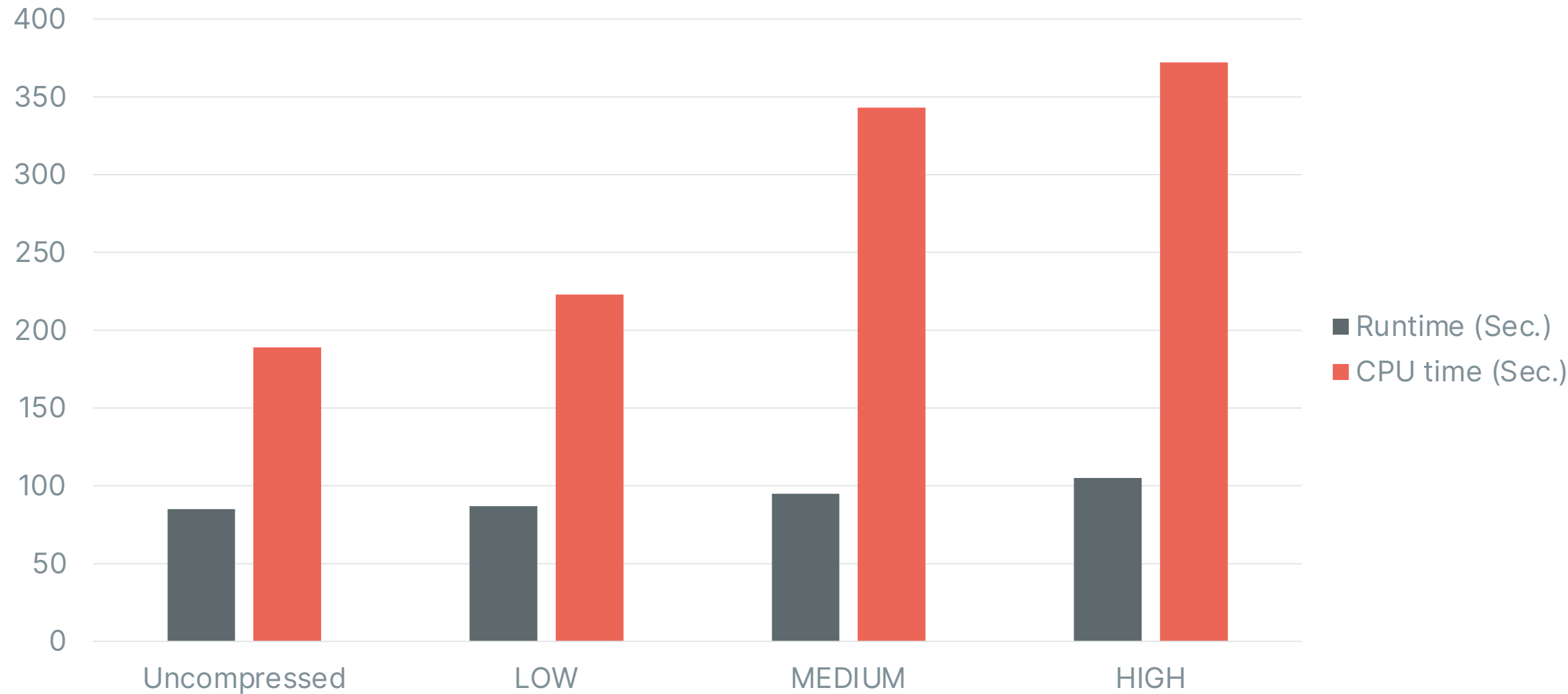
Beispiel LOB-Komprimierung

- Beispiel: JSON-Daten in CLOB mit massiv Redundanz in den Feldbezeichnern (0,5 .. 100 KB, durchschnittlich 3,5 KB)
- Migration einer nicht partitionierten und nicht komprimierten Tabelle
- Ziel-Struktur:
 - Tabelle intervall-partitioniert nach Zeit, ca. 30 Partitionen
 - LOB mit den JSON-Dokumenten als Securefile und komprimiert
- CREATE TABLE .. PARALLEL AS SELECT /*+ PARALLEL */ ... parallelisiert nicht mit LOB
- Statt dessen:
 - CREATE TABLE <new> ... LOB (c) STORE AS SECUREFILE (ENABLE STORAGE IN ROW COMPRESS);
 - ALTER SESSION ENABLE PARALLEL DML;
 - INSERT /*+ APPEND PARALLEL(64) */ INTO <new> SELECT /*+ PARALLEL(64) */ FROM <old>;
- Messwerte für die ersten 1 Mio. Records

LOB: Objekt-Größen nach Kompression partitoned Table

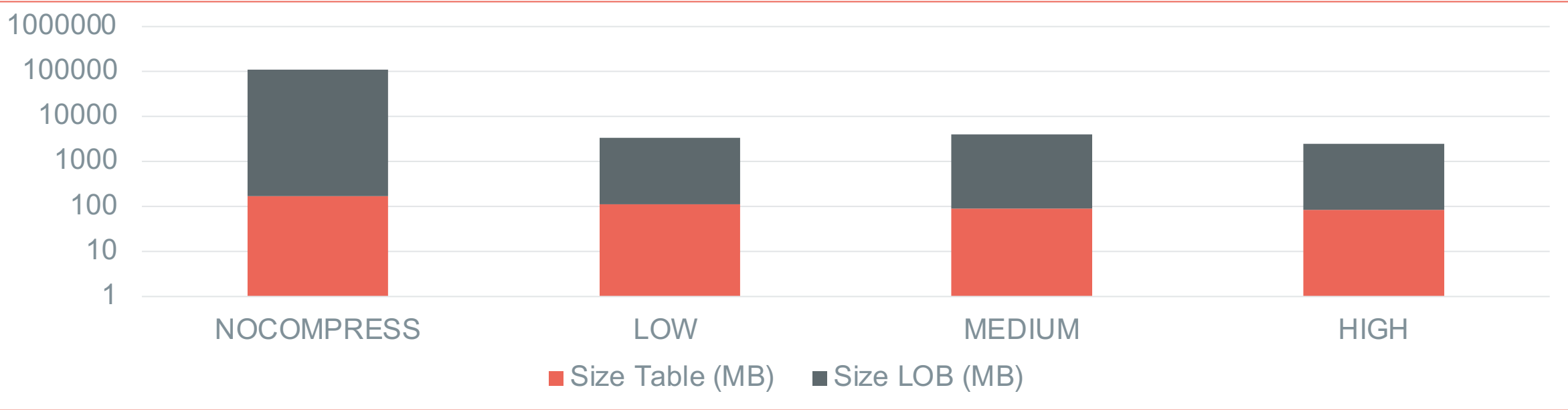
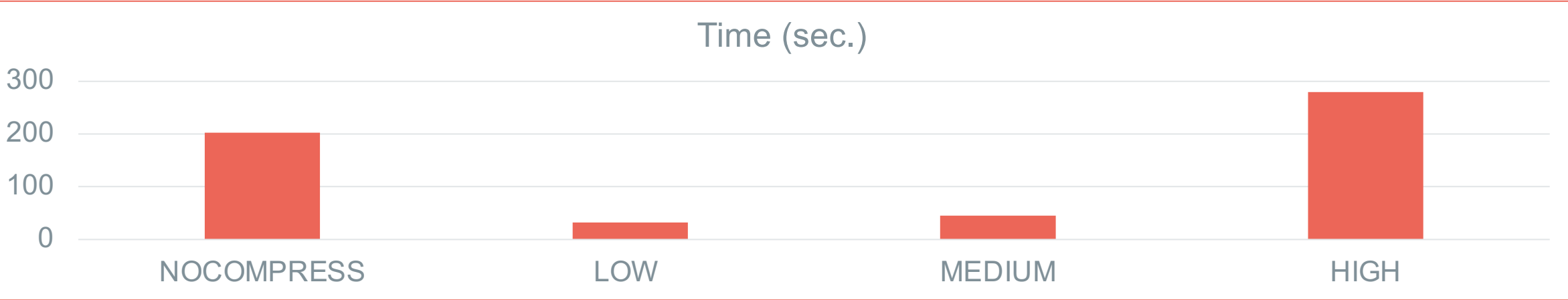


LOB: Insert-Zeiten für 1 Mio. Records partitioned Table (parallel)



Anderes Beispiel: LOB mit JSON Daten (Faktor 46:1 bei HIGH)

ALTER TABLE MOVE LOB(A) STORE AS SECUREFILE(xx) PARALLEL;



Generieren von Vorschlägen für Komprimierung

- DBMS_COMPRESSION.Get_Compression_Ratio soll zu erwartende Komprimierung ermitteln
- Noch nie selbst erfolgreich benutzt, läuft auf diverse Fehler in "SYS.PRVT_COMPRESSION"
- Das Analysetool ‚Panorama‘ enthält einige Vorschlagslisten für Tabellen- und Index-Komprimierung

Rasterfahndung nach Performance-Bottlenecks und Nutzung von Anti-Pattern

Auswahl des Rasterfahndungs-SQL

Filter: Include description ☒ Search

1. Potential in DB-Strukturen

1. Sicherstellen optimaler Storage-Parameter für Indizes

2. Empfehlungen für Index-Komprimierung, Test auf Selektivität

3. Empfehlungen für Index-Komprimierung, Test auf Leaf-Blocks

4. Empfehlungen für Index-Komprimierung, Test auf Selektivität einer einzelnen Spalte eines Mehrspaltenindex

5. Empfehlungen für ADVANCED HIGH Index-Komprimierung

8. Empfehlungen für evtl. sinnvolle OLTP-Komprimierung von Tabellen

9. Evtl. nicht sinnvolle OLTP-Komprimierung von Tabellen

17. Evtl. komprimierbare bislang unkomprimierte LOB-Segmente

Empfehlungen für evtl. sinnvolle OLTP-Komprimierung von Tabellen

Tabellen-Komprimierung (COMPRESS FOR xxx) reduziert I/O-Aufwände durch Verbesserung der Cache-Hit-Rate.
Reduzierung der Größe der Tabellen durch Komprimierung um 1/3 bis 1/2 möglich.
Min. 20% Reduktion der Größe sowie Relevanz von I/O sollte vorhanden sein, um CPU-Overhead für Komprimierung/Dekomprimierung zu kompensieren.

Die OLTP-Komprimierung ist gut nutzbar für Tabellen mit Insert- und Delete-Operationen.
Während Update-Operationen findet Dekomprimierung der DB-Blöcke statt mit evtl. Chained Rows, daher sollten für OLTP-Komprimierung nur wenige oder gar keine Update-Operationen auf der Table stattfinden.

OLTP-Komprimierung erfordert die Lizenzierung der Advanced Compression Option.

Mindestgröße der Tabelle oder Partition in MB

10

Max. % Updates verglichen zu Inserts + Deletes

5

Minimale Anzahl Tage seit der letzten Analyse

7

Selektion ausführen

Show SQL

Empfehlungen für evtl. sinnvolle OLTP-Komprimierung von Tabellen: 'Mindestgröße der Tabelle oder Partition in MB' = 10, 'Max. % Updates verglichen zu Inserts + Deletes' = 5, 'Minimale Anzahl Tage seit der letzten Analyse' = 7

owner	table_name	partitions to compress	partitions total	num_rows	size_mb	max_last_analyzed	inserts	updates	deletes	last_dml
CUST	DOUBLEACCOUNTCHECKRESULT	16	16	49.070.963	34.331,00	01.04.2023 20:41:21	1.119.888	131.852	4.455.956	10.05.2023 02:02:12
AUFTRAG	AU_LZ_LOOKUP			496.471.539	19.686,38	26.02.2023 02:30:33				
PATCH	EKINF_SEASON_HIST			300.105.931	17.868,00	01.05.2023 08:13:08				
CUST	CUSTTOPICGROUP	16	16	173.287.556	9.210,00	29.04.2023 20:12:40	700.361	35.681	11.690.421	10.05.2023 02:02:12
LOGISTIK	LO_RET_MIGRATION_RECHPOS			125.141.627	7.472,00	19.03.2023 02:56:20				
Menü anzeigen	AU_WG_UMSATZ_ORIGINAL	1	2	36.111.954	7.384,00	05.12.2021 02:52:37				

- | Overview over objects in tablespaces | | | | | | | | | | | | | | | | | | | | | | | | | |
|--|----------|------------------|---------------------|-----------|--------------------|-----------------------------------|----------------|--------------|------------|-------------|-----------|-----------------------------------|---------------------|---------------------|---------------------|---------------|-------------|-----------------|--------------|------------|---------------------|---------------------|---------------------|----------------------------------|------------------|
| Tablespaces
[All] | | | Schemata
AUFTRAG | | | Part.
☐ | | | Filter
 | | | Show objects and sizes | | | | | | | | | | | | | |
| Objects for Tablespace='[All]' Schema='AUFTRAG' | | | | | | | | | | | | | | | | | | | | | | | | | |
| TS | Schema | Name | Part. | Type | Compr. | Init. ext. (KB) | Next ext. (KB) | Min. ext. | Used ext. | Ini. trans. | Pct free | MBytes | No. of rows | Avg. row len. | % unused | MBytes unused | Comp. ratio | MBytes uncompr. | Blocks empty | Avg. Space | Created | Last DDL time | Last sp. time | | |
| DATASP01 | AUFTRAG | AU_BEW_TMP_SST_1 | 1 | TABLE | ENABLED (ADVANCED) | 64 | 1,024 | 1 | 97 | 8 | 0 | 130 | 4,766,696 | 152 | | | 5.7 | 745 | 0 | 0 | 2019-12-08 02:07:43 | 2023-09-03 03:11:11 | 2019-12-08 02:07:43 | | |
| INDEXSP01 | AUFTRAG | FK_KONTOAUSZU_KI | 1 | INDEX | ADVANCED | 64 | 1,024 | 1 | 566 | 8 | 10 | 47 | 17,669,214 | 2 | | | 4.9 | 231 | | | 2016-11-13 | 2024-02-07 | 2016-11-13 | | |
| Description of TABLE AUFTRAG.AU_BEW_TMP_SST_175_ERR | | | | | | | | | | | | | | | | | | | | | | | | | |
| "DML Error Logging table for ""AU_BEW_TMP_SST_175""" | | | | | | | | | | | | | | | | | | | | | | | | | |
| Columns of TABLE AUFTRAG.AU_BEW_TMP_SST_175_ERR | | | | | | | | | | | | | | | | | | | | | | | | | |
| Col.Name | Type | Prec. | Sc. | N. | Def. | Distinct | Nulls | Avg. Len. | Density | Buckets | Histogram | Comments | | | | | LOB segment | EQ | EQJ | NEQJ | Range | Like | Null | | |
| ORA_ERR_NUMBER\$ | NUMBER | 22 | | Y | | 0 | 4,766,696 | 0 | 0.0000 | 0 | NONE | ORA Fehlernummer | | | | | | | | | | | | | |
| ORA_ERR_MESG\$ | VARCHAR2 | 2000 | | Y | | 3 | 0 | 43 | 0.3333 | 1 | NONE | ORA Fehlertext | | | | | | | | | | | | | |
| ORA_ERR_ROWID\$ | UROWID | | | Y | | | | | | | | Rowid, bei der der Fehler auftrat | | | | | | | | | | | | | |
| Attributes of TABLE AUFTRAG.AU_BEW_TMP_SST_175_ERR | | | | | | | | | | | | | | | | | | | | | | | | | |
| TS | Pct Free | Init. Trans | Initial extent (KB) | Rows | Size (MB) Table | Size (MB) Total | Ext. | Compr. ratio | Row len. | Part. | Sub-Part. | Created | Last DDL | Spec. TS | Last analyzed | Log. | Mon. | Inserts | Updates | Deletes | Trunc. | Drop seg. | Last DML | Several States | RAC master inst. |
| DATASP01 | 0 | 8 | 64 | 4,766,696 | 129.75 | 129.75 | 97 | 5.7 | 152 | 0 | 0 | 2019-12-08 02:07:43 | 2023-09-03 03:11:11 | 2019-12-08 02:07:44 | 2023-12-17 02:34:39 | NO | YES | | | | | | | Compression = ENABLED (ADVANCED) | 2 |

Bewertung des Ergebnis der Komprimierung

- Mittels DBMS_COMPRESSION.Get_Compression_Type kann die Art der Komprimierung je RowID ermittelt werden
- Damit kann detailliert verfolgt werden, welche Zeilen der Tabelle wie komprimiert sind
- Panorama zeigt dies auf Basis von Stichproben in wählbarer Genauigkeit (Klick auf „Compression Ratio“)

Attributes of TABLE AUFTRAG.AU_BEW_TMP_NOA_NALI_ANSPR

TS	Pct Free	Init. Trans	Initial extent (KB)	Rows	Size (MB) Table	Size (MB) Total	Ext.	Compr. ratio	Row len.	Part.	Sub-Part.	Created	Last DDL	Spec. TS	Last analyzed	Log.	Mon.	Inserts	Updates	Deletes	Trunc.
DATA01	< 2 different >	8	8,192	7,282,644	616.50	856.88	346	3.1	31	62	0	2020-02-02 02:12:57	2024-02-28 06:10:03	2024-02-28 00:21:58	2024-02-28 00:21:57		YES	316,187	0	238,064	NO

1 Indexes

1 Primary Key

0 Check Constraints

0 References from

0 References to

Visual Refs.

0 Triggers

9 Dependencies

13 Grants

DBMS_METADATA

DB-Cache

Sessions accessing

SQLs

Active Session History

Segment statistics

Size evolution

Check compression state for AUFTRAG.AU_BEW_TMP_NOA_NALI_ANSPR

Gap between considered rows Show compression state

Real compression state per DB block for AUFTRAG.AU_BEW_TMP_NOA_NALI_ANSPR

Checked blocks	Checked rows	Compression type
4,225	4,225	No Compression
2,025	71,481	COLUMN STORE COMPRESS FOR ARCHIVE HIGH

- Komprimierungsfunktionen der Oracle-DB sind sinnvoll produktiv einsetzbar
- Je nach Anwendungsprofil kann sich neben der Storagereduktion ein Laufzeitvorteil oder Nachteil ergeben
- Erfordert teilweise individuelle Bewertung und Tests auf Objekt-Ebene
- Teilweise aber auch Nobrainer,
z.B. Advanced Index Key Compression könnte auch pauschal auf Tablespace-Ebene aktiviert werden
- Advanced Compression Option kann sich lohnen (ca. 1/4 der EE-Preise)
- Problem bei Lizenzierung der Advanced Compression Option
 - Die Funktionen und Benefits der Komprimierung sind oftmals Software-Entwicklern unbekannt
 - Damit zwar die Option lizenziert, aber nicht genutzt an Stellen wo deutliche Effekte möglich wären

Weitere Ressourcen zu Oracle Compression

- Panorama: Performance-Analyse von Oracle-DB: <https://rammpeter.github.io/panorama.html>
- [Database Compression Resources \(Gregg Christman\)](#)
- [19c Oracle Advanced Compression, White Paper 02/2019](#)
- [Oracle Advanced Compression Proof-of-Concept Guidelines, Insights and Best Practices 21.2](#)
- [Automatic Data Optimization \(ADO\), White Paper 19c](#)

Vielen Dank für Ihr Interesse

Otto Group Solution Provider (OSP) Dresden GmbH
Freiberger Str. 35 | 01067 Dresden
T +49 (0)351 49723 0 | F +49 (0)351 49723 119
[osp.de](https://www.osp.de)